

Ubuntu Server Command Reference

For Django projects (Daphne / Gunicorn, Nginx, Celery, Redis, PostgreSQL, systemd, Git) — FavHost

Service Management (systemctl)

```
systemctl status <service-name>
```

Shows whether a service is running, since when, and its last few log lines. Your first command for any "is it broken" question.

```
sudo systemctl start <service-name>
```

Starts a service right now. Does not affect whether it starts automatically on the next reboot.

```
sudo systemctl stop <service-name>
```

Stops a running service immediately.

```
sudo systemctl restart <service-name>
```

Stops then starts a service — use after changing code or config that the service needs to reload.

```
sudo systemctl daemon-reload
```

Tells systemd to re-read service files after you've created or edited one. Required before a new/edited .service file will take effect.

```
sudo systemctl enable <service-name>
```

Sets a service to auto-start every time the server boots. Does not start it immediately by itself.

```
sudo systemctl disable <service-name>
```

Removes a service from auto-starting on boot.

```
systemctl list-units --type=service --all | grep -i <keyword>
```

Finds the real name of a service when you're not sure what it's called (e.g. searching "gunicorn" or "favhost").

Logs & Debugging

```
journalctl -u <service-name> -f
```

Live-tails a service's logs in real time — watch it work (or fail) as it happens. Ctrl+C to stop watching.

```
journalctl -u <service-name> --since "15 minutes ago" --no-pager
```

Shows recent history for a service without live-following — good for reviewing what happened a few minutes ago.

```
journalctl -xe
```

Shows the most recent system-wide log entries with extra context — useful when you don't know which service is at fault yet.

```
sudo tail -n 50 /var/log/nginx/error.log
```

Shows Nginx's own error log — the first place to check for 502/504 gateway errors.

```
sudo tail -n 50 /var/log/nginx/access.log
```

Shows recent incoming requests Nginx has handled.

```
sudo dmesg | grep -i "killed process"
```

Checks the kernel log for out-of-memory (OOM) kills — confirms if the kernel force-killed a process due to low RAM.

System Health (Memory, Disk, CPU)

```
free -h
```

Shows total/used/free/available RAM and swap, in human-readable units. Check this whenever the site feels slow.

```
df -h
```

Shows disk space usage across all mounted drives.

```
df -h /
```

Shows disk space for the root partition specifically — check before creating large files like a swap file.

```
uptime
```

Shows how long the server's been running plus the "load average" — numbers well above your CPU core count mean the CPU is overloaded.

```
top / htop
```

Live, continuously updating view of running processes and their CPU/memory usage. htop is a nicer-looking version (may need installing).

```
du -sh <folder>
```

Shows the total size of a specific folder — useful for finding what's eating disk space.

Process Inspection

```
ps aux | grep <name>
```

Lists running processes matching a keyword — quick way to check if something (e.g. redis, celery) is actually alive.

```
sudo ss -tlnp | grep <port>
```

Shows what process, if any, is listening on a specific network port.

```
kill <PID>
```

Terminates a process by its ID. Prefer systemctl stop for anything managed by systemd — kill is for one-off manual processes.

Redis

```
redis-cli ping
```

Quick health check — returns PONG if Redis is running and reachable.

```
redis-cli info memory
```

Shows how much memory Redis itself is currently using.

```
redis-cli monitor
```

Live-streams every command Redis receives in real time — useful for watching Celery tasks arrive as they happen.

```
redis-cli flushall
```

Deletes everything stored in Redis. Only safe if Redis holds no data you care about (e.g. only used as a Celery queue, not caching).

Celery

```
celery -A bookaid status
```

Checks whether any Celery workers are alive and responding. Run from inside the project folder with the venv active.

```
celery -A bookaid inspect active
```

Shows tasks currently being processed right now.

```
celery -A bookaid inspect scheduled
```

Shows tasks that Celery beat has scheduled to run soon.

```
celery -A bookaid inspect registered
```

Lists every task type the worker knows about — useful to confirm your task functions were picked up correctly.

Django (manage.py)

```
python manage.py runserver
```

Starts Django's built-in dev server. Local development only — never used in production (that's Daphne/Gunicorn's job).

```
python manage.py migrate
```

Applies pending database migrations to update the schema.

```
python manage.py makemigrations
```

Generates new migration files based on changes to your models.

```
python manage.py shell
```

Opens an interactive Python shell with your Django project already loaded — handy for quick data checks.

```
python manage.py collectstatic
```

Gathers all static files (CSS/JS/images) into one folder for production serving via Nginx.

```
python manage.py createsuperuser
```

Creates a Django admin account interactively.

```
python manage.py dbshell
```

Drops you directly into your database's own command-line shell (psql, for PostgreSQL).

PostgreSQL

```
sudo -u postgres psql
```

Logs into PostgreSQL as the superuser account.

```
\l
```

Lists all databases (run inside the psql prompt).

```
\c <dbname>
```

Connects to a specific database (inside psql).

```
\dt
```

Lists all tables in the currently connected database (inside psql).

```
\q
```

Exits the psql prompt.

Git

```
git status
```

Shows changed, staged, and untracked files in your current branch.

```
git log --oneline -10
```

Shows the last 10 commits in condensed, one-line-each form.

```
git diff
```

Shows exactly what's changed but not yet committed.

```
git pull origin <branch>
```

Fetches and merges the latest changes from a remote branch.

```
git checkout <branch>
```

Switches to a different branch.

```
git stash
```

Temporarily shelves uncommitted changes so you can switch branches cleanly, restorable later with git stash pop.

Virtual Environment & Packages

```
source env/bin/activate
```

Activates the project's Python virtual environment — isolates its packages from the rest of the system.

```
deactivate
```

Exits the currently active virtual environment.

```
pip list
```

Shows all Python packages installed in the active environment.

```
pip freeze > requirements.txt
```

Writes the currently installed package versions into requirements.txt.

```
sudo apt update
```

Refreshes Ubuntu's package lists. Always run before installing something new.

```
sudo apt install <package> -y
```

Installs a system-level package (e.g. redis-server), auto-confirming the prompt.

General troubleshooting order: whenever something on the server seems broken, check things in this sequence — **systemctl status** (is it running?) → **journalctl logs** (why did it fail?) → **free -h / df -h** (is it a resource problem?) — before taking any action. This order alone resolves most day-to-day server issues.